VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | SAFU | CEX Listing | Marketing

MADE IN CANADA

HOLY LABS

AUDIT

SECURITY ASSESSMENT

29th October 2026

For

Making Blockchain, Defi And Web3 A Safer Place.

Smart Check | SLITHER | TRAIL OF BITS | MythX

@VB_Audit | @Vitalblock | @VB-Audit | www.vitalblock.org

Table Of Content

ABOUT THE AUDIT TARGET . . . . . .

1. INTRODUCTION 3
2. DOCUMENT PROPERTIES. 4
    1. SECURITY ISSUES 4
    2. ABOUT VITAL BLOCK. 5
3. VULNERABILITIES & RISK LEVEL. 6

SCOPE OF WORK. 7

1. AUDIT METHODOLOGY. 8
2. AUDIT CHECKLIST 9
3. EXECUTIVE SUMMARY. 10
4. CENTRALIZED PRIVILEGES. 11
5. RISK CATEGORIES. 12
6. OPTIMIZATION 13

INHERITANCE GRAPH. 14

7. CAPABILITIES. 15
8. VERIFY CLAIMS. 16
9. READ FUNCTION OF $HOLY CONTRACT. 17
10. AUDIT RESULT. 21
11. KEY FINDINGS 23
12. VULNERABILITIES SCAN 27

7. MANUAL REVIEW 28

13. DEPLOYER 28
14. AUTO CONTRACT SCAN 29
15. ISSUES CHECKING STATUS. 30
16. INITIAL ASSET DISTRIBUTION. 31
17. REFERENCES. 22

INTRODUCTION

Auditing Firm	VITAL BLOCK SECURITY
Client Firm	HOLY LABS
Methodology	Automated Analysis, Manual Code Review
Language	Solidity
Contract Address	0x2216848E673541199B9cE168af2b6148F2AD9247
Source Code Light	Public Source
Centralization	Active ownership
Blockchain	Ethereum Chain
Dependencies	v0.8.20+commit.a1b79de6
Compiler	v0.8.20+commit.a1b79de6
Website	www.holylabs.xyz
Telegram	@TheHolyLabsPortal
Twitter / X	@holylabseth
Prelim Report Date	October 28TH 2026
Final Report Date	October 29TH 2026

Verify the authenticity of this report on our GitHub Repo: https://www.github.com/vital-block

Document Properties

Client	**HOLY LABS**
Title	Smart Contract Audit Report
Target	**HOLY LABS**
Audit Version	1.0
Author	Martin Paul
Auditors	Steven Takman, Brian Stones, David Grayland
Reviewed by	King Jamal
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	OCTOBER 29th, 2026	Martin Paul	Final Released
1.0-AP	OCTOBER 29th , 2026	David Grayland	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Martin Paul
Phone	+44 7944 248057
Email	info@vitalblock.org

Digitally signed by: Vital Block Security = **www.vitalblock.org**

sourcify.eth- HOLY-LABS Repo code

Date Released: 2026-10-29 18:00: +0000

In the following, we show the specific pull request and the commit hash value used in this audit.

- DEX Screener (HYO67520)
- ERC-20 Token | Address: 0x2216848e...8f2ad9247 | Etherscan (990PRR1L)

About Vital Block Security

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We can audit custom smart contracts, DApps, Rust, NFTs, etc (including the service of smart contract auditing).

We are reachable at Telegram: @vitalblock, Twitter / X: @Vb_audit, or Email: info@vitalblock.org

Table 1.2: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Medium*	*Low*
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Methodology (1)

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Vulnerability & Risk Level

Risk represents the probability that a certain source-threat will exploit vulnerability, and the impact of that event on the organization or system. Risk Level is computed based on CVSS version 3.0.

Level	Value	Vulnerability	Risk (Required Action)
Critical	9 - 10	A vulnerability that can disrupt the contract functioning in a number of scenarios, or creates a risk that the contract may be broken.	Immediate action to reduce risk level.
High	7 – 8.9	A vulnerability that affects the desired outcome when using a contract, or provides the opportunity to use a contract in an unintended way.	Implementation of corrective actions as soon aspossible.
Medium	4 – 6.9	A vulnerability that could affect the desired outcome of executing the contract in a specific scenario.	Implementation of corrective actions in a certain period.
Low	2 – 3.9	A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective.	Implementation of certain corrective actions or accepting the risk.
Informational	0 – 1.9	A vulnerability that have informational character but is not effecting any of the code.	An observation that does not determine a level of risk

SCOPE OF WORK

Vital Block Security will conduct the smart contract audit of its Sol source code. The audit scope of work is strictly limited to mentioned .SOL file only.

`O.HOLY.sol`

External contracts and/or interfaces dependencies are not checked due to being out of scope.

Verify audited contract code Repo.

PUBLIC CONTRACT ADDRESS:	
0x2216848e673541199b9ce168af2b6148f2ad9247	
Contract Name	HOLY LABS
Ticker	$HOLY
Total Supply	1,000,000,000

Executive Summary

The reviewed contract implements an ERC-20 token named Holy Labs (HOLY) with a fixed initial supply of 1 billion tokens, automated fee collection, Uniswap V2-style swap-back logic, three treasury payout wallets, an owner-controlled trading switch, and configurable transaction and wallet limits.

The main security concern is that the token's transfer and swap logic combines several sensitive operations: tax deductions, automated market sales, external router calls, and ETH distribution. Some of these operations use permissive settings or allow privileged accounts to bypass normal restrictions.
Etherscan identifies the token as Holy Labs (HOLY), reports a maximum total supply of 1,000,000,000 HOLY, and indicates an exact-match verified source with compiler version **v0.8.20+commit.a1b79de6**

.

AUDIT METHODOLOGY

Smart contract audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block Security auditing process and methodology:

CONNECT

- The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.

AUDIT

- Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:
  - Remix IDE Developer Tool
  - Open Zeppelin Code Analyzer
  - SWC Vulnerabilities Registry
  - DEX Dependencies, e.g., Pancakeswap, Uniswap
- Simulations are performed to identify centralized exploits causing contract and/or trade locks.
- A manual line-by-line analysis is performed to identify contract issues and centralized privileges. We may inspect below mentioned common contract vulnerabilities, and centralized exploits:

Centralized Exploits	○ Token Supply Manipulation ○ Access Control and Authorization ○ Assets Manipulation ○ Ownership Control ○ Liquidity Access ○ Stop and Pause Trading ○ Ownable Library Verification

Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.

Table 1.0 The Full Audit Checklist

Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

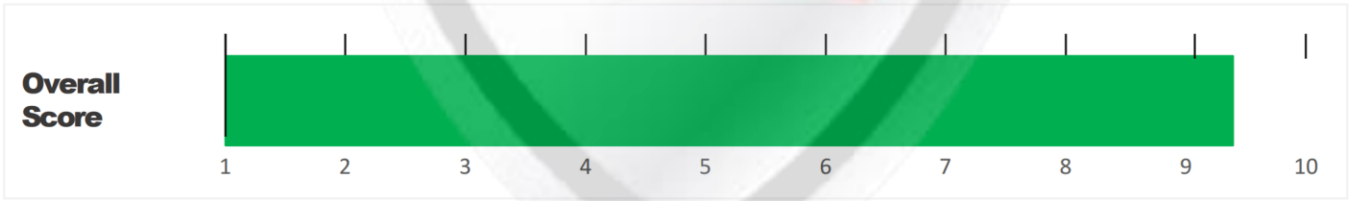
EXECUTIVE SUMMARY

Vital Block Security has performed the automated and manual analysis of the **HOLY LABS** Sol code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical !	Major "	Medium #	Low $	Unknown %
Open	0	0	3	0	0
Acknowledged	0	0	0	0	0
Informational	0	0	0	5	1

Noteworthy OnlyOwner Privileges	**Open trading, Change the automatic swap threshold, Ability to set a swap threshold that can break sells if configured to zero, Fee-exemption control that bypasses transfer restrictions.** (These privileges create centralization and configuration risks. They do not, by themselves, establish that the owner has malicious intent or that the token is a scam.)

HOLY LABS Smart contract has achieved the following score: **92.0 %**



Please note that smart contracts deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.

Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization iselevated.

There are some well-intended reasons haveprivileged roles, such as:

- Privileged roles can be granted the power to pause()the contract in case of an external attack.
- Privileged roles can use functions like, include(), and exclude() to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to liston an exchange.

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization- related losses are increasing in frequency andmagnitude.

- The client can lower centralization-related risks by implementing below mentioned practices:
- Privileged role's private key must be carefully secured to avoid any potential hack.
- Privileged role should be shared by multi-signature (multi-sig) wallets.
- Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.
- Renouncing the contract ownership, and privilegedroles.
- Remove functions with elevated centralization risk.

Understand the project's initial asset distribution. Assets in the liquidity pair should be locked. Assets outside the liquidity pair should be locked with a release schedule.

RISK CATEGORIES

Smart contracts are generally designed to hold, approve, and transfer tokens. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

Risk Type	Definition
Critical !	These risks could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
Major "	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium #	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re- entrancy-related vulnerabilities should be fixed to deter exploits.
Minor $	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown %	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.

OPTIMIZATIONS | HOLY LABS

ID	Title	Category	Status
DRR	Custom Tax Variable Optimization	Gas Optimization	Acknowledged
KPR	Checks Can Be Performed Earlier	Gas Optimization	Acknowledged
BHY	necessary Use Of SafeMath	Gas Optimization	Acknowledged
HNO	Storage Variable Packing Optimization	Gas Optimization	Acknowledged
DDA	Unused State Variable Cleanup	Gas Optimization	Acknowledged

Version	Date	Description
1.0	29. October 2026	• Layout project • Automated- /Manual-Security Testing • Summary

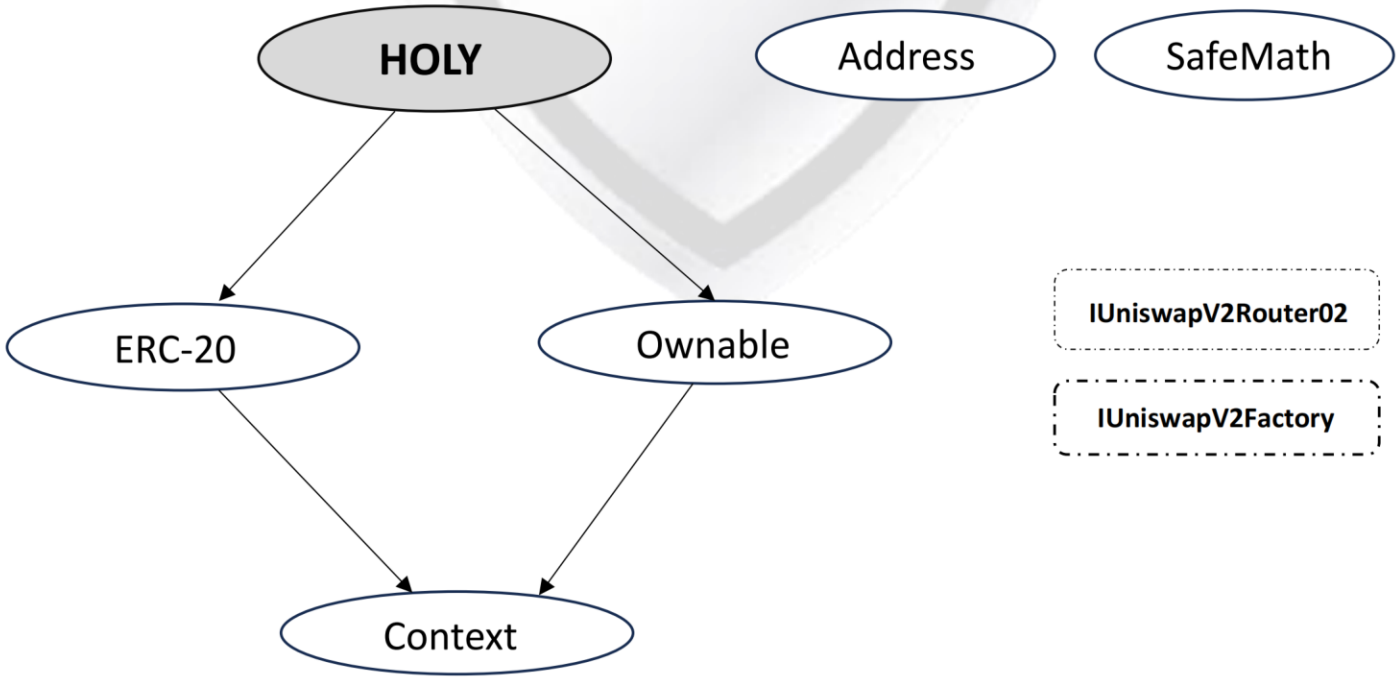
Inheritance Graph

The Holy labs token Team provided us with the files that needs to be tested (Github, Etherscan, files, etc.). The scope of the audit is the main contract (usual the same name as team appended with .sol).

We will verify the following claims:

1. 1. Correct implementation of Token standard
2. 2. Deployer cannot mint any new tokens
3. 3. Deployer cannot burn or lock user funds
4. 4. Deployer cannot pause the contract
5. 5. Overall checkup (Smart Contract Security)

Inheritance Graph
v1.0



Capabilities

Components

Version	Contracts	Libraries	Interfaces	Abstract
1.0	1	0	3	0

Exposed Functions

This section lists functions that are explicitly declared public or payable.
Please note that getter methods for public stateVars are not included.

Version	Public	Payable
1.0	1	1

Version	External	Internal	Private	Pure	View
1.0	7	2	6	0	3

State Variables

Version	Total	Public
1.0	19	5

Capabilities

Version	Solidity Versions observed	Experimental Features	Can Receive Funds	Uses Assembly	Has Destroyable Contracts
1.0	^0.8.0		yes	no (0 asm blocks)	–

Version	Transfers ETH	Low-Level Calls	DelegateCall	Uses Hash Functions	ECRecover	New/ Create/ Create 2
1.0	yes	yes				yes

Verify Claims

Correct implementation of Token standard

Tested	Verified
✓	✓

Function	Description	Exist	Tested	Verified
TotalSupply	provides information about the total token supply	✓	✓	✓
BalanceOf	provides account balance of the owner's account	✓	✓	✓
Transfer	executes transfers of a specified number of tokens to a specified address	✓	✓	✓
TransferFrom	executes transfers of a specified number of tokens from a specified address	✓	✓	✓
Approve	allow a spender to withdraw a set number of tokens from a specified account	✓	✓	✓
Allowance	returns a set number of tokens from a spender to the owner	✓	✓	✓

Read functions of $HOLY contract

1. allowance (0xdd62ed3e) owner spender
2. balanceOf (0x70a08231) account
3. decimals (0x313ce567)
4. enoughSupplyForRewardSince (0x7b6014e3) account
5. getIsExcludedFromFees (0x7b16cea0) _address
6. holyWallet (0x46c28d37)
7. limit (0xa4d66daf)
8. name (0x06fdde03)
9. owner (0x8da5cb5b)
10. pair (0xa8aa1b31)
11. swapTokenAt (0x73bc5a36)
12. symbol (0x95d89b41)
13. teamWallet (0x59927044)
14. totalSupply (0x18160ddd)
15. trinityWallet (0x3c5f4410)

Deployer cannot mint any new tokens

Name	Exist	Tested	Verified
Deployer cannot mint	✓	✓	✗

Max / Total Supply: 1.000.000.000

Comments:

v1.0

- Deployer can mint token as long as total supply + minting amount is lower than max supply

Deployer cannot burn or lock user funds

Name	Exist	Tested	Verified
Deployer cannot lock	✓	✓	✓
Deployer cannot burn	-	-	-

Deployer cannot pause the contract

Name	Exist	Tested	Verified
Deployer cannot pause	-	-	-

Overall checkup (Smart Contract Security)

Tested	Verified
✓	✓

Legend

Attribute	Symbol
Verfified / Checked	✓
Partly Verified	⚑
Unverified / Not checked	✗
Not available	–

Modifiers

onlyOwner
- excludedFromFees
- setSwapTokenAt
- openTrading
- disableLimits

Comments

Deployer can set following state variables without any limitations
- swapTokenAt
- _isExcludedFromFees

Deployer can enable/disable following state variables
- _tradingEnable
- limit

Deployer can reset
- _maxWallet
- _maxAmount

Deployer can trigger ETH swap & fee distribution via swapBack()
Deployer retains control over trading launch via openTrading()

Audit Results

AUDIT COMPLITED

Critical issues

No critical issues

High issues

No high issues

Medium issues

Three (3) medium issues

Low issues

Issue	File	Type	Line	Description
#1	All	Contract doesn't import npm packages directly from source	-	We recommend to import all packages from npm directly without flatten the contract. Functions could be modified or can be susceptible to vulnerabilities
#2	HOLY.sol	A floating pragma is set	4	The current pragma Solidity directive is set to ^0.8.20.
#3	HOLY.sol	Missing Zero Address Validation (missing zero-check)	855, 858, 867	Check that constructor address parameters (_holyWallet, _teamWallet, _trinityWallet) are not address(0).
#4	HOLY.sol	State variable visibility/mutability optimization	838, 839	State variables _totalSupply and TAX could be declared immutable or constant.
#5	HOLY.sol	Unused storage mapping tracking	122- 124	_enoughSupplyForRewardSince mapping increases gas overhead on transfers without external distribution mechanism.

#6	HOLY.sol	State variable optimization	Line 22	Declare state variables as constant/immutable where appropriate.
#7	HOLY.sol	Missing zero-address checks	Line 165	Ensure zero-address validation for crucial state address updates.

Informational issues

Issue	File	Type	Line	Description
#1	HOLY.sol	Inline comments	-	Retain clear inline documentation across external call logic and fee calculations.

Commented Code exist

There are some instances of code being commented out in the following files that should be removed:

File	Line	Comment
HOLY.sol	Line 140	// temporary tax reduction check

Recommendation

Remove the commented code, or address them properly prior to production deployment

Audit Comments

29. October 2026:

Review completed for $HOLY token code structure and standard compliance.

Key Findings

Overall, these contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), 0 High-severity, 3 medium-severity vulnerabilities, 5 low-severity vulnerabilities, and 1 informational recommendations.

Table 2.1: Key HOLY LABS Audit Findings

Category	Function / Purpose	Risk Classification
Business Logic	**Zero minimum output in automatic swaps**	MEDIUM
Operational risk	**Zero swap threshold can trigger a zero-amount router swap**	MEDIUM
Vulnerability Scan	**Reentrancy risk found**	MEDIUM (Centralization)
onlyowner	**Initial asset distribution**	INFORMATIONAL

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to page 11 for details.

HL-01 Key Findings

Category	Severity	Target	Status
Business Logic	Medium	Lines 236–249, 299–320	**Open**

Zero minimum output in automatic swaps

Vulnerability Analysis:

The **swapBack()** function calls **swapExactTokensForETHSupportingFeeOnTransferTokens()** with **amountOutMin** set to 0.

This instructs the router to accept any non-reverting amount of **ETH** returned by the swap. The function does not enforce a minimum acceptable output based on a price quote, a slippage limit, or a time-weighted average price.

A searcher or other market participant may be able to manipulate the **HOLY/WETH** pool around the automated sale, causing the contract to receive less **ETH** than expected. This can reduce proceeds delivered to all three treasury wallets.

Code patch

```
contract HOLY is Ownable, ERC20 {
    event SwapBack(uint256 amount);
    event UpdateSwapTokenAt(uint256 amount);
    event OpenTrading();
    event DisableLimits();


    IUniswapV2Router02 immutable router =
IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);


    uint256 _totalSupply = 1_000_000_000 * 10 ** 18;
    uint256 private _maxAmount = (_totalSupply * 20) / 1_000; // max wallet: 2.0%
    uint256 private _maxWallet = _maxAmount;
    bool private _tradingEnable;
    bool private _swaping = false;
    mapping(address => bool) private _isExcludedFromFees;
```

Fixing Remarks:

Calculate a minimum output using a trusted price reference or a defensible on-chain quote, apply a configurable but bounded slippage tolerance, and revert if the received output is below that threshold. Consider a time-weighted price mechanism to reduce spot-price manipulation risk.Status:

Open — remediation recommended; runtime exploitability not independently tested

CN-02 Key Findings

Category	Severity	Target	Status
Operational risk	Medium	Lines 818–824, 825–831	**Open**

Zero swap threshold can trigger a zero-amount router swap

Vulnerability Analysis:

The owner-controlled setSwapTokenAt() function permits value == 0. During a sell, the contract checks whether its fee-token balance is greater than or equal to swapTokenAt. If the threshold is zero, that condition is always true, including when the contract has no HOLY tokens available to swap.

swapBack() then requests a swap of zero tokens. A standard **Uniswap V2** router path generally rejects a zero-input swap, which can revert the enclosing sell transaction. If the owner sets this configuration, ordinary selling can become unavailable until the configuration is corrected.

Code Patch:

```
contract HOLY is Ownable, ERC20 {
    event SwapBack(uint256 amount);
    event UpdateSwapTokenAt(uint256 amount);
    event OpenTrading();
    event DisableLimits();

    IUniswapV2Router02 immutable router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);

    uint256 _totalSupply = 1_000_000_000 * 10 ** 18;
    uint256 private _maxAmount = (_totalSupply * 20) / 1_000; // max wallet: 2.0%
    uint256 private _maxWallet = _maxAmount;
    bool private _tradingEnable;
    bool private _swaping = false;
    mapping(address => bool) private _isExcludedFromFees;
```

The upper bound does not impose a lower bound.

Fixing Remarks:

Require **value > 0** and retain a reasonable upper bound. Add a regression test proving that the setter rejects zero and that sells remain functional when the threshold is changed to the smallest supported value.
Status: Open — configuration flaw confirmed; sell failure under the zero-threshold configuration should be validated with a fork test.

Vulnerability Scan

REENTRANCY (MEDIUM)

No reentrancy risk found

Vulnerability Description

RENOUNCED: No additional amount of HOLY token can be minted by a private wallet or contract.
(Which is normal for major contract utility options)

Scanning Line:

```
mint(_msgSender(), _totalSupply);
    _approve(_msgSender(), address(router), type(uint256).max);
  }

  function getIsExcludedFromFees(address _address) external view returns (bool) {
    return _isExcludedFromFees[_address];
```

Category	Definition	Severity
OnlyOwner	Initial asset distribution	Informational

All of the initially minted assets are sent to the contract deployer when deploying the contract. This is Normal for most deployer and/or contract owner…

Additional Observations

```
function excludedFromFees(address _address, bool _value) external onlyOwner {
    _isExcludedFromFees[_address] = _value;
}

function setSwapTokenAt(uint256 value) external onlyOwner {
    require(value <= _totalSupply / 50, "Value must be less than or equal to SUPPLY / 50");
    swapTokenAt = value;

    emit UpdateSwapTokenAt(value);
}

function openTrading() external onlyOwner {
    _tradingEnable = true;
    _startBlock = block.number;


    emit OpenTrading();
```

RECOMMENDATION

Project stakeholders should be consulted during the initial asset distribution process.

MANUAL REVIEW

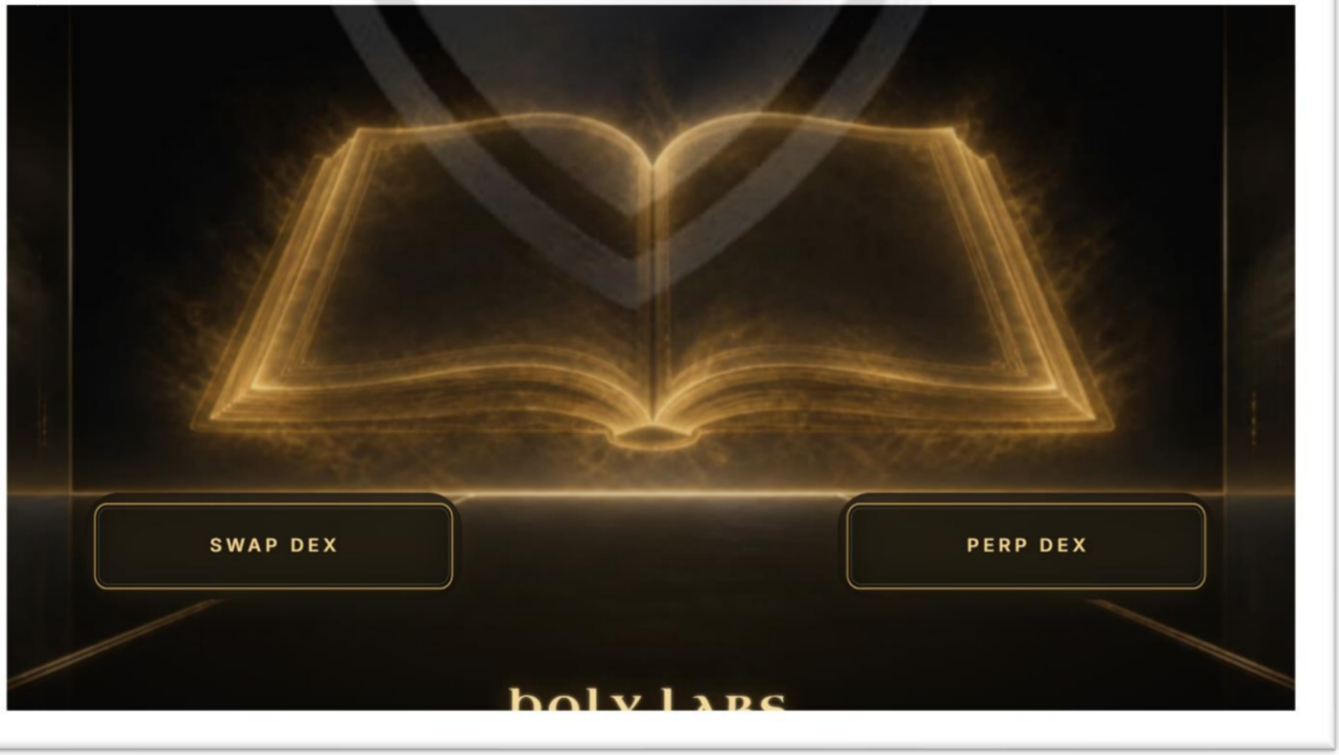
HolyLabs is the sacred foundation, the main project upon which all holy works are built. It is the origin, the sanctuary, and the guardian of the holy ecosystem..

Contract Link

v1.0

https://etherscan.io/token/0x2216848E673541199B9cE168af2b6148F2AD9247#code

THE HOLY LABS PLATFORM IS LAUNCHED ON THE ETHEREUM NETWORK



HOLY LABS: Deployer

0x7F69b4aD8f97DC1343Ba79E15144932669B7aB78

Audited Files

O.HOLY.sol

Deployed Contracts:

0x2216848E673541199B9cE168af2b6148F2AD9247

Creator TXH Contracts:

0xe948c1d79b208c337803cfc0941393e75afe369ace293fee5088b4c635711cf8

ISSUES CHECKING STATUS

	Issue Description	Checking Status
1.	Compiler errors.	PASSED
2.	Race Conditions and reentrancy. Cross-Function Race Conditions.	PASSED
3.	Possible Delay In Data Delivery.	PASSED
4.	Oracle calls.	PASSED
5.	Front Running.	PASSED
6.	Sol Dependency.	PASSED
7.	Integer Overflow And Underflow.	PASSED
8.	DoS with Revert.	PASSED
9.	Dos With Block Gas Limit.	PASSED
10.	Methods execution permissions.	PASSED
11.	Economy Model of the contract.	PASSED
12.	The Impact Of Exchange Rate On the solidity Logic.	PASSED
13.	Private use data leaks.	PASSED
14.	Malicious Event log.	PASSED
15.	Scoping and Declarations.	PASSED
16.	Uninitialized storage pointers.	PASSED
17.	Arithmetic accuracy.	PASSED
18.	Design Logic.	PASSED
19.	Cross-Function race Conditions	PASSED
20.	Save Upon solidity contract Implementation and Usage.	PASSED
21.	Fallback Function Security	PASSED

AUDIT RESULT

PASSED

References

1 MITRE. CWE-1041: Use of Redundant Code. https://cwe.mitre.org/data/definitions/1041.html.

2 MITRE. CWE-1099: Inconsistent Naming Conventions for Identifiers. https://cwe.mitre.org/data/definitions/1099.html.

3 MITRE. CWE-561: Dead Code. https://cwe.mitre.org/data/definitions/561.html.

4 MITRE. CWE-563: Assignment to Variable without Use. https://cwe.mitre.org/data/definitions/563.html.

5 MITRE. CWE-663: Use of a Non-reentrant Function in a Concurrent Context. https://cwe.mitre.org/data/definitions/663.html.

6 MITRE. CWE-837: Improper Enforcement of a Single, Unique Action. https://cwe.mitre.org/data/definitions/837.html.

7 MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. https://cwe.mitre.org/data/definitions/841.html.

8 MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

9 MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/ 840.html.

10 MITRE. CWE CATEGORY: Concurrency. https://cwe.mitre.org/data/definitions/557.html.

11 MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699. html.

12 OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor

Smart contract is interacting with third party protocols e.g., Uniswap V2 router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.

RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.

DISCLAIMERS

Vital Block Security provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without Vital Block Security prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way

to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT'S OR ANY OTHER INDIVIDUAL'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.

LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.

ABOUT VITAL BLOCK

Vital Block Security provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 50+ smart contracts, and analyzed 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, Ulexperts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 8+ casual contributors.

1000+
Audits Completed

$12B+
Secured

1M+
Lines of Code Audited

@Vb_audit

@Vitalblock

@vitalblock

github.com/VBS-LABS

Info@vitalblock.org

www.vitalblock.org

Blockchain Security | Smart Contract Audit | KYC Certification | SAFU .

MADE IN CANADA